

Varp Manual

Tony Rogvall

2026-09-16

Contents

Introduction	1
Getting started	2
The language	2
Variables and declarations	2
Expressions	3
Quantifiers	3
Macros	3
Circuits	3
Assignments and the top level formula	4
Running varp	5
Plugins	5
Global options	5
Backjump options	5
Transition systems and model checking	6
The bmc plugin	7
Several systems, channels and instances	7
Synthesis	8
The GUI	8
Appendix: files and formats	8

Introduction

Varp is a SAT based theorem prover and constraint solver. A problem is written in the varp language: propositional logic over named variables, quantifiers that unroll over integer ranges, bit vector arithmetic, macros, circuits and transition systems. Everything is compiled down to clauses and handed to a CDCL solver written in C, and the answer comes back as models (assignments), a count, a proof, or a counterexample trace.

The same formula can be evaluated, solved or synthesised, since varp makes no difference between a known and an unknown value: a lookup table with a known configuration computes a function, the same table with the configuration left free is programmed by the solver.

This manual describes the language, the command line, the plugins and the model checking extension. SYNTAX.md is the grammar reference, doc/CIRCUIT.md and doc/MODEL_CHECKING.md go deeper into circuits and transition systems, and formulas/varp/ holds the examples used here.

Getting started

Varp needs Erlang/OTP (with wx for the GUI) and a C compiler.

```
make          # the Erlang code and the NIF
make test     # the test suite
varp --help   # or priv/varp.sh
```

A first formula, the pigeonhole principle:

```
// n pigeons do not fit in n-1 holes
([A p=1..n] [E h=1..(n-1)] P(p,h)) and
([A h=1..(n-1)] [A p=1..n] [A q=p+1..n] not (P(p,h) and P(q,h)))
```

```
$ varp sat bj n=5 formulas/varp/pigeon.varp
% 0
```

% 0 means no model, the formula is unsatisfiable. A satisfiable formula prints its models and the count:

```
$ varp sat bj n=6 formulas/varp/die_hard.varp
1: B(0)=0,B(1)=5,B(2)=2,...,Fill_big(1),Big_to_small(2),...
% 1
```

The general form of a command line is

```
varp [global options] [plugin [plugin options]]* [bindings] [files]
```

Global options come first, plugin options follow their plugin, a binding is name=value and gives a value to a meta variable such as the n above, and the files are read in order. varp <plugin> -h prints the options of a plugin.

The language

Variables and declarations

A propositional variable springs into existence when it is used: $P(p,h)$ above is a family of variables indexed by two integers. Vectors, integers of a given width, are declared:

```
declare X:8, Y:8/signed;    // 8 bit unsigned and signed
declare B(i):4;             // a family of 4 bit vectors
```

Widths may be meta variables, declare $X:n$. The option `--undeclared` controls what happens when a name looks like a misspelling of another one.

Expressions

The logic operators are `not`, `and`, `or`, `xor`, `implies` (or `imp`) and `equ` (equivalence), with the C spellings `!`, `&&`, `||` also accepted. Vectors support the arithmetic and bitwise operators of C: `+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `~`, shifts `<<`, `>>` and rotations `<<<`, `>>>`, comparisons `==`, `!=`, `<`, `<=`, `>`, `>=`, bit selection `X[i]`, and the conditional `c ? a : b`, which is a bitwise multiplexer when the branches are vectors. Arithmetic on signed vectors sign extends, and a vector assigned to a narrower one is cut at the top. `min` and `max` are built in.

Integer constants are decimal, hex `0x1021`, octal or binary `0b101`.

Quantifiers

A quantifier unrolls its body over integer ranges of meta variables:

```
[A i=1..n] P(i)           // for all i
[E i=1..n] P(i)           // for some i
[E! i=1..n] P(i)          // for exactly one i
[EQ 2](a, b, c, d)        // exactly two of the list
[SUM i=1..n] X(i)         // and other counting forms
```

`[ALL ...]`, `[ANY ...]`, `[ONE ...]`, `[NONE ...]`, `[PARITY ...]`, `[ODD ...]`, `[EVEN ...]` are the spelled out forms, and a quantifier may bind several variables at once, `[A i=1..n, j=1..m]`.

A quantifier takes the *next primary expression* as its body, so a comparison is parenthesised: `[A p=0..n-1] (st(p) == 0)`. Written without the parentheses, `[A p=0..n-1] st(p) == 0` compares the quantifier to zero and leaves `p` free.

Inside a quantifier body the bound variables are integers and may appear in index expressions, `B((p+1) % n)`, in comparisons with vectors, `who == p`, and in meta level tests between each other, `(f != p) implies ...`, which fold to constants.

Macros

```
define NEXT(i) [EQ 1](Fill_small(i), ...) and (FILL_SMALL(i) or ...);
define SEED 0xffff;
```

A `define` is a macro with integer parameters, expanded where it is called. The parameters are integers such as a step or an index, not formulas: to abstract over variables, write a circuit. A macro without parameters is a named constant.

Circuits

A circuit is a reusable block with parameters, a closed scope of its own, and a value:

```
circuit half_adder(in y, z; return x; out co)
{
    x = y ^ z;
    co = y && z;
}
```

```
circuit lut3(in cfg:8; in x, y, z; return q)
{
    q = (cfg[0] && !x && !y && !z) || ... || (cfg[7] && x && y && z);
}
```

in parameters are read, out parameters are assigned by the body and land in the caller's variables, and the return parameter is the value of the call, so a circuit can be used inside an expression: `z = rca(x, y, false, cout)`. Local variables are declared in the body, every instance gets its own copies (`rca#1.c`), and a body may contain plain formulas as constraints.

That last point is what generates structures: a quantified circuit call in a body instantiates the circuit once per index, which is how an `n` bit adder is built from full adders:

```
circuit fa(in a, b, ci; out s, co)
{
    s = a ^ b ^ ci;
    co = (a && b) || (ci && (a ^ b));
}

circuit rca(in a:n, b:n, ci; return s:n; out co)
{
    declare S(i), C(i);
    C(0) = ci;
    [A i=0..n-1] fa(a[i], b[i], C(i), S(i), C(i+1));
    [A i=0..n-1] (s[i] equ S(i));
    co = C(n);
}
```

`doc/CIRCUIT.md` has the details: scope, defaults, named arguments, nested circuits and the errors.

Assignments and the top level formula

A file is a list of definitions (`declare`, `define`, `circuit`, `system`), then assignments `x = expr`;, then one formula. Assignments are constraints that hold in every model, the formula is what is asked about; a file with only assignments asks whether they are consistent.

Running varp

Plugins

A command line names a chain of plugins, each with its options. The mode plugins decide the question, the search plugins answer it:

plugin	short	what it does
satisfy	sat	find models of the formula
falsify	unsat	find models of its negation
prove	p	show that the formula is valid, % TRUE/% FALSE
backtrack	bt	DPLL search
backjump	bj	CDCL search with learning, restarts, minimisation
bmc	bmc	bounded model checking of a system, see below
order	ord	the initial variable order
saturate	s	probing of one or two literals at a time
cnf, validate, monitor, wx		export, model checking, progress, the GUI

--max <N> (-n) on the search plugin is the number of models to find, 0 for all.

Global options

The ones you reach for most, varp --help prints them all:

```
--qtype lifo|fifo|recursive  propagation order          (lifo)
--bump-decay <f>              VSIDS activity decay, 0 = list (0.95)
--use-phase <bool>            phase saving                  (false)
--phase true|false|undefined  initial phase                  (true)
--timeout <s>                  give up after s seconds
--log info|debug               progress and statistics
--print model|erlang|dimacs|false  how models are printed
--undeclared none|typo|once|all    warnings about names
```

Backjump options

```
backjump --max-learned <L>          learned clause limit          (0)
          --max-learned-factor <F>    L = F * |clauses|              (0)
          --max-learned-inc <F>        growth per purge              (0)
          --keep-factor <P>            fraction kept on a purge      (0.5)
          --minimize none|local|recursive (none)
          --restart-counter <N>         restart every N bcp calls    (0)
          --restart-interval <s>        restart every s seconds
          --bump <N>|none|...          only none matters with decay
```

With the defaults learning is unlimited and restarts are off. A recipe that does well on arithmetic problems such as factoring:

```
varp --qtype=fifo --use-phase=true sat bj --minimize recursive \
  --max-learned-factor 2 --max-learned-inc 1.2 --keep-factor 0.5 \
  --restart-counter 100000 n=412351270399 formulas/varp/is_prime.varp
```

varp_tune searches such recipes automatically over a ladder of instances, see TUNING.md.

Transition systems and model checking

A system describes a machine: state, inputs, initial states and transitions, and a property. bmc unrolls it step by step and looks for a run that reaches the property, or proves that none does.

```
system jugs {
  state B:4, L:4;
  input fill_small, fill_big, empty_small, empty_big,
        small_to_big, big_to_small;

  init  B == 0 and L == 0;

  next  [EQ 1](fill_small, fill_big, empty_small, empty_big,
              small_to_big, big_to_small) and
        (fill_small implies (next(L) == 3 and next(B) == B)) and
        (fill_big   implies (next(B) == 5 and next(L) == L)) and
        ...;

  reach B == 4;
}
```

```
$ varp bmc formulas/varp/die_hard_system.varp
bmc: counterexample at k=6
  step  B  L  input
      0  0  0
      1  5  0  fill_big
      2  2  3  big_to_small
      ...
      6  4  3  big_to_small
% 1
```

Inside next a state variable is its current value and next(X) the value in the following step. The items of a system:

item	meaning
state <decls> ;	the state variables, typed like declare, may be indexed
input <decls> ;	free choices made in every step
init <expr> ;	the initial states
next <expr> ;	the transition relation
assume <expr> ;	holds in every step, a constraint on state and input

item	meaning
reach <expr> ;	some reachable state satisfies it
invariant <expr> ;	every reachable state satisfies it
eventually <expr> ;	bounded liveness, refuted by a loop that avoids it
send <ch> <expr> [when <cond>] ;	drive the sending end of a channel
recv <ch> <state> [when <cond>] ;	drive the receiving end of a channel

A system is definitions, not a mode: it exports `jugs_init(i)`, `jugs_next(i)`, `jugs_reach(k)` and `jugs(k)`, and a file may use them in a formula of its own, `varp sat bj k=6 ...` runs one bound.

The bmc plugin

```
varp bmc [--k-min 0] [--k-max 20] [--step 1] [--property <macro>]
        [--induction] [bj <options>] <file>
```

The bound is raised until a counterexample appears (% 1 with the trace), or k-max is passed (% 0). The clause database is kept between bounds (--incremental), so learned clauses carry over. --induction proves an invariant by k-induction: % TRUE when the step case has no model, % FALSE with a trace when the base case has one, % UNKNOWN past k-max. Deadlock freedom of the ordered dining philosophers is proved this way with a strengthening invariant:

```
varp bmc --induction --property dining_invariant n=5 formulas/varp/dining_correct.varp
% TRUE
```

Several systems, channels and instances

Systems in one file compose synchronously and share variables by name: a system writes `next(x)`, another reads `x`. A channel is a queue between systems, a system of its own with the slots and the count as state, driven by `send` and `recv` statements:

```
channel ch:4[2];

system producer(dst) {
    state v:4;
    init  v == 1;
    send  dst v when v <= 3;
    next  (v <= 3) implies next(v) == v + 1;
    next  (v > 3) implies next(v) == v;
}

system consumer(src) {
    state got:4, sum:4;
    input take;
    init  sum == 0 and got == 0;
```

```

    recv  src got when take and src_n > 0;
    next  (take and src_n > 0) implies next(sum) == sum + src_q0;
    next  (not (take and src_n > 0)) implies next(sum) == sum;
    reach sum == 6;
}

```

```

instance p = producer(ch);
instance c = consumer(ch);

```

A system with parameters is a template, instance copies it with the parameters bound and its other names prefixed (`p_v`, `c_sum`). The dining philosophers in `formulas/varp/dining_msg_dead.varp` are three philosophers and three forks as instances talking over twelve channels; bmc finds the deadlock at $k=4$.

Synthesis

Leaving part of a structure free turns evaluation into synthesis. The configuration of a lookup table is its truth table, so the configuration of a target function is the target evaluated on every row:

```

circuit target(in x, y, z; return q) { q = !x && y && z; }
define TT(m) target((m & 1) == 1, (m & 2) == 2, (m & 4) == 4);
declare cfg:8;

```

```
[A m=0..7] (cfg[m] equ TT(m))           // cfg = 64
```

`formulas/varp/fpga_fabric.varp` goes further: four cells with routing multiplexers, and the solver programs both the configurations and the routing so that the net adds two 2 bit numbers, in under a second.

The GUI

```
varp --gui=true           # or priv/varp_gui.sh
```

The window has the formula on the left, models on the right, and buttons Satisfy, Falsify and BMC. BMC takes its bound from “k max”, proves by k-induction when “Induction” is ticked, and prints the verdict and the trace table in the model window. Profiles hold the search options.

Appendix: files and formats

Input files are `.varp` (the language), `.cnf` (DIMACS) and `.snf` (DIMACS with symbolic names). `varp cnf` exports a formula as DIMACS. `formulas/varp/` holds the examples of this manual, `test/` the test suite (`make test`, `make test-gui`), and `test/bench.sh` a raw performance benchmark.